UNIVERSIDAD DE LAS FUERZAS ARMADAS

INNOVACIÓN PARA LA EXCELENCIA

DCCO

Second Semester

Workshop 14 Project Inspection

OOP - 23217

Professor: Edison Lascano

2025-05-24

RUBRIC

TOPIC	EVAL	OBSERVATIONS
1. Definition	/10	
2. SRS	/10	11:44 - 11:54
3. Class and UC Diagrams	/10	
4. CSV (10 files)	/10	
5. Mockup/maquetado o diseño de la IU	/10	5 screens / c/u = 2
6-7. Clean Code	/20	pitfall -1, other pitfall occurrence only note, int a; -1 inc names int b; inc name … let a; -1 inc names let b; inc names
8-9-. Functionality	/20	10 requirements for validation assign appointments → it works → 1
10. persistence (consistency)	/10	verification
TOTAL	/100	

EVIDENCE

1. Definition
2. SRS
3. Class and UC Diagrams
4. ER (10 tables)/CSV (10files)
5. Mockup
6-7. Clean Code
8-9-. Functionality
10. persistence (consistency)

✔️ Team 1: SoftCrafters

Project: Eduplan
Leader: Andrade Julio
Github: https://github.com/julitobby/ESPE2504-OOPSW23217-SoftCrafters.git

Inspector Team 4: JSnow, Inspector Leader: Sivinta Jahir

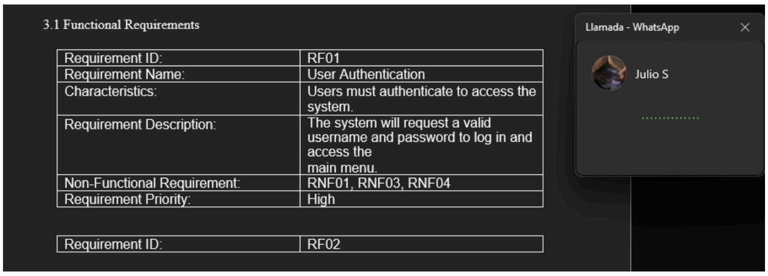
1. Andrade Julio
2. Gerald Astudillo
3. Bonilla David
4. Caizapanta Tammy

RUBRIC

TOPIC	EVAL	OBSERVATIONS
1. Definition	8.5/10	The document is well written but lacks background information. Talk more about the problem.
2. SRS	8/10	It has 10 functional requirements, but the non-functional requirements are the interface and data management in the cloud.
3. Class and UC Diagrams	8/10	Generalize the requirements. Too many requirements. Some are the same as others and use verbs and missing cases in the case diagrams
4. ER (10 tables)/CSV (10files)	6/10	Does not have the required 10 csv, but the ones they have are very well structured.
5. Mockup	8.7/10	The mockup has the idea, but it's ambiguous It’s a great idea for mckup but a better presentation it’s more professional.
6-7. Clean Code	16/20	The code is very well structured, although it has some problems with class names, and some comments that could be reduced, but the code complies with the clean code standard.

8-9-. Functionality	17/20	The program works very well, although in some parts it is a little difficult to understand how the system works, it is quite functional and does its job, although it should be more solid.
10. persistence (consistency)	9/10	The program saves files correctly but lacks organization. The menu is overcrowded and file storage lacks a dedicated folder, causing minor disorder.
TOTAL	81.2/100	

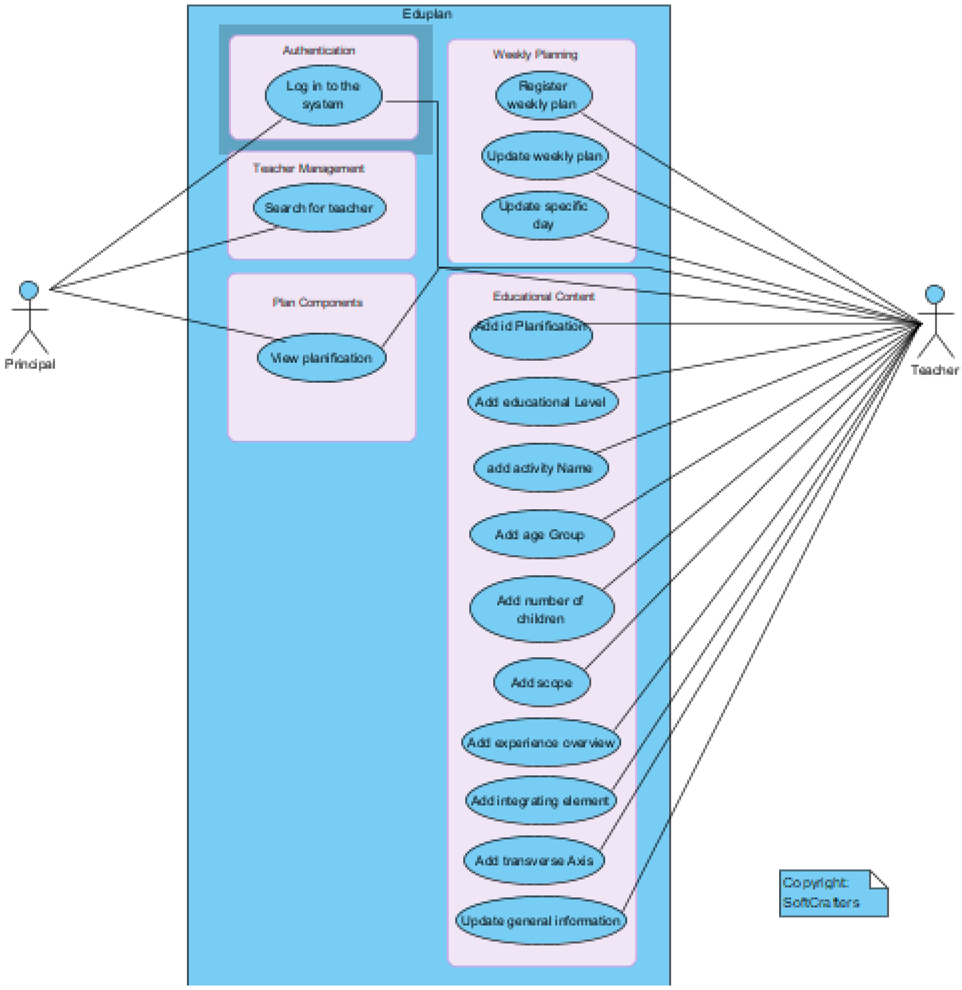
EVIDENCE

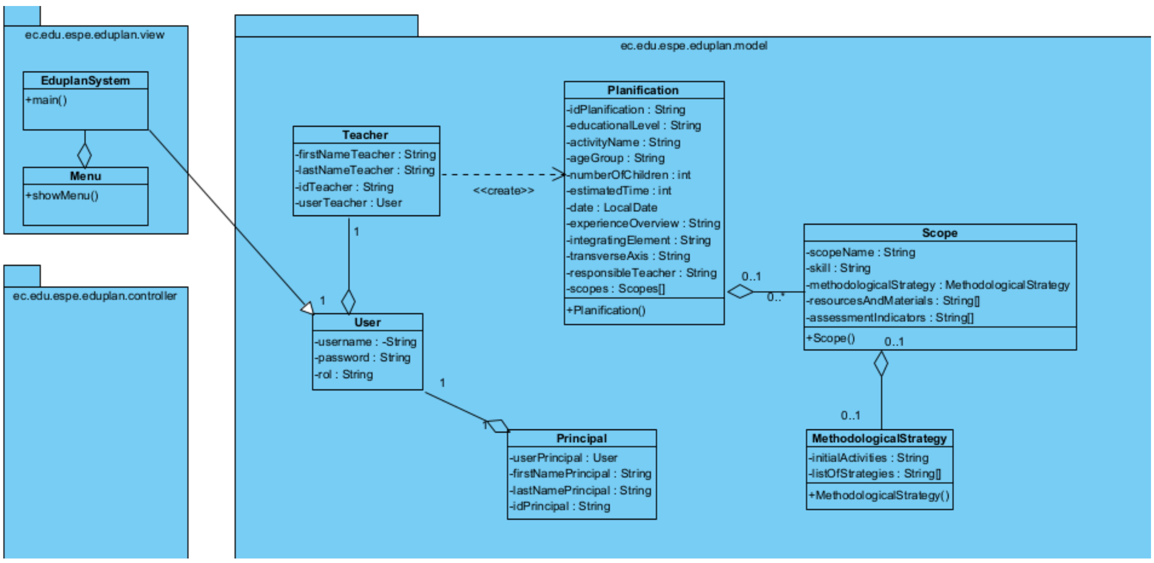


3.1 Functional Requirements

Requirement ID:	RF01
Requirement Name:	User Authentication
Characteristics:	Users must authenticate to access the system.
Requirement Description:	The system will request a valid username and password to log in and access the main menu.
Non-Functional Requirement:	RNF01, RNF03, RNF04
Requirement Priority:	High

Requirement ID:	RF02





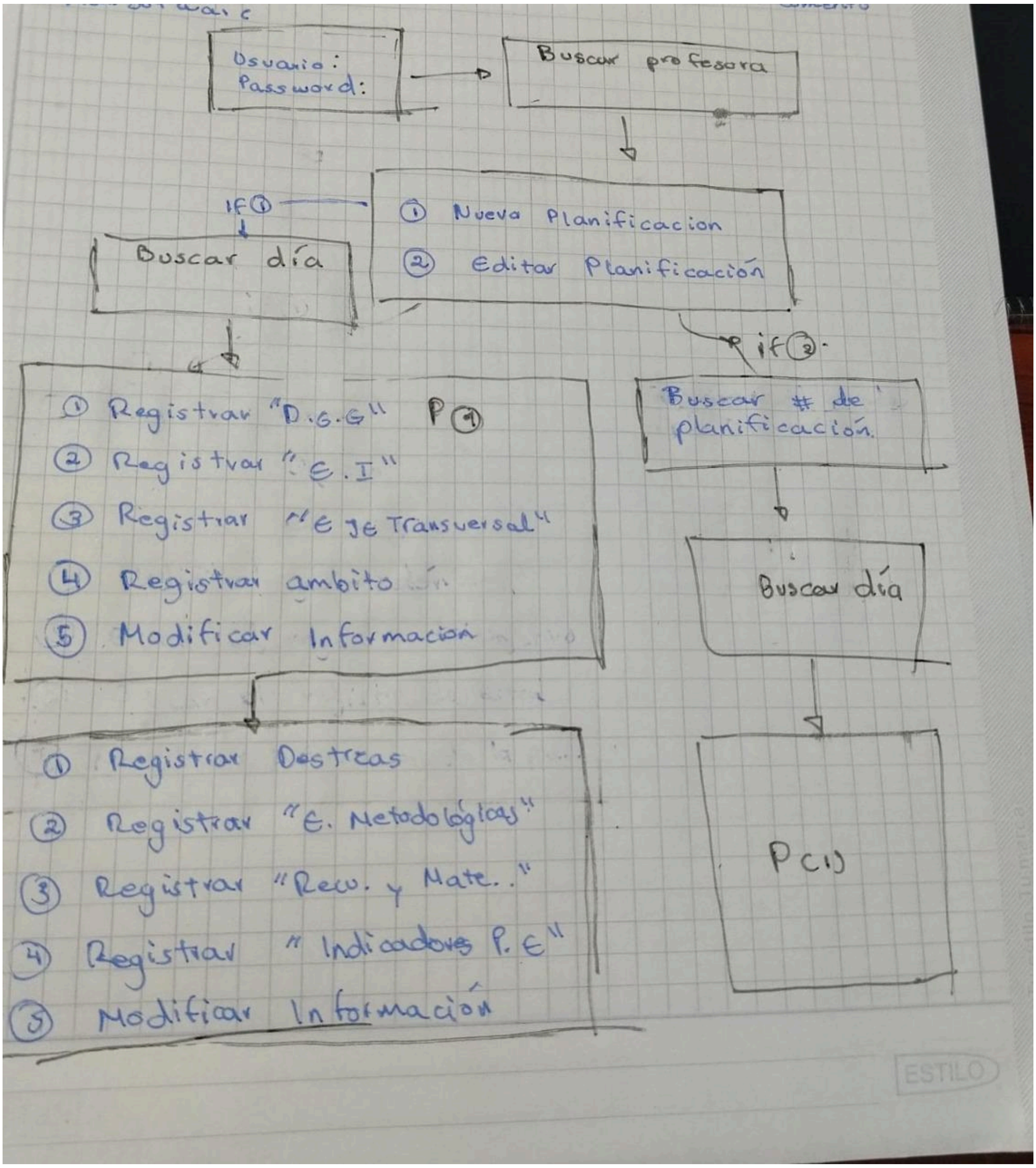
4. System Features

1. FR001

Requirement ID	FR001
Requirement Name	Log in
Type	Functional Requirement
Actor	Director / Teacher
Requirement Description	Develop a secure login interface requiring users to provide credentials for system access.
Inputs	Username, Password
Outputs	Authentication
Requirement Priority	High

2. FR002

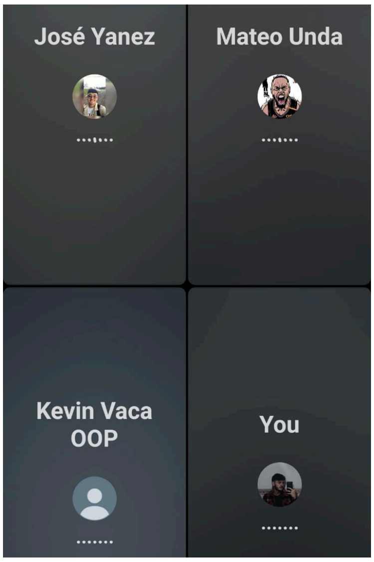
Requirement ID	FR002
Requirement Name	User Registration
Type	Functional Requirement
Actor	Director / Teacher (New User)
Requirement	Implement a robust user registration module allowing new



1. Definition
2. SRS
3. Class and UC Diagrams
4. ER (10 tables)/CSV (10files)
5. Mockup
6-7. Clean Code
8-9-. Functionality
10. persistence (consistency)

✔️ Team 2: Black Mesa

Project: Craft_Market
Leader: Micaela Jácome
Github:
https://github.com/MicaelaJacomeCalahorrano/ESPE2504-OOPSW23217-Craft_Market.git
Inspector Team: TEAM 5 Yánez, Suarez, Unda, Vaca



1. Michael Chicaiza.
2. Jorge Fuentes
3. Micaela Jácome
4. Isaac Maisincho

RUBRIC

TOPIC	EVAL	OBSERVATIONS
1. Definition	8/10	The document defines their project in a great way, but there are missing concepts in the background. A cover page might improve the document.
2. SRS	8.5/10	It follows the IEEE structure, it is easy to understand and keeps coherence. Section 3 fully meets the requirements, but Sections 1 and 2 are incomplete according to IEEE standards.
3. Class and UC Diagrams	6.5/10	UC: it isn't created by using verbs, so these use cases aren't valid. Class: There are 10 classes, but only the classes of the model are with associations, and the

		classes of view are without relations.
4. ER (10 tables)/CSV (10files)	6/10	The CSV files are in many folders, it's difficult to find them, and it's not well distributed, also there are only six CVS files
5. Mockup	8.5/10	Very clear and legible, it's easy for a external user understand the structure and options that the program has to offer, and also this menu has the name of the user that is seeing the menu, a good utility.
6-7. Clean Code	17/20	The code is well structured, the name of the methods are good but can be better, the name of the variables are good and the code is understandable
8-9-. Functionality	18/20	The code works well, some things didnt work like are supposed to work but it didnt give an error.
10. persistence (consistency)	8.5/10	CSV persistence works, but files lack folder organization. A structured directory and consistent naming would improve maintainability.
TOTAL	81/100	

EVIDENCE

1. Definition

Overview

Referring to the sales of artisan,we would focus on each product and its variants for each artisan selling in the market since many of them have similar products and prices, we will also take into account each artisan's attendance, the time they arrive and start their shift at the market, and their departure time.

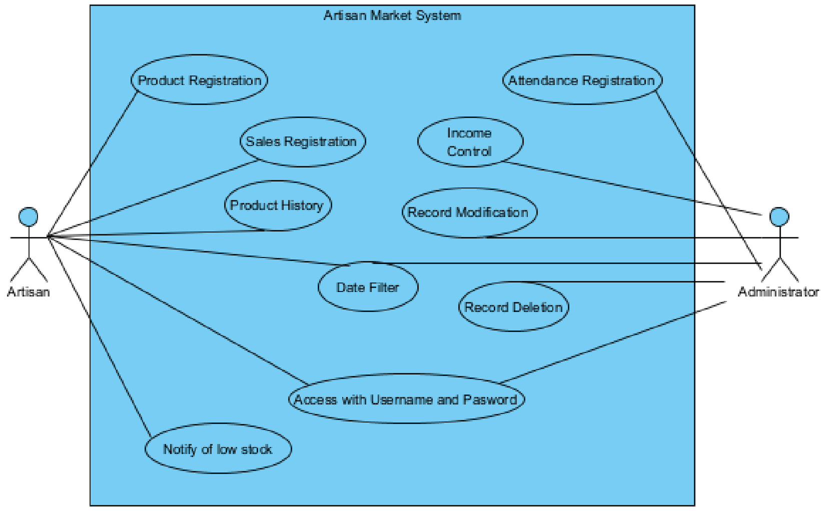
Background

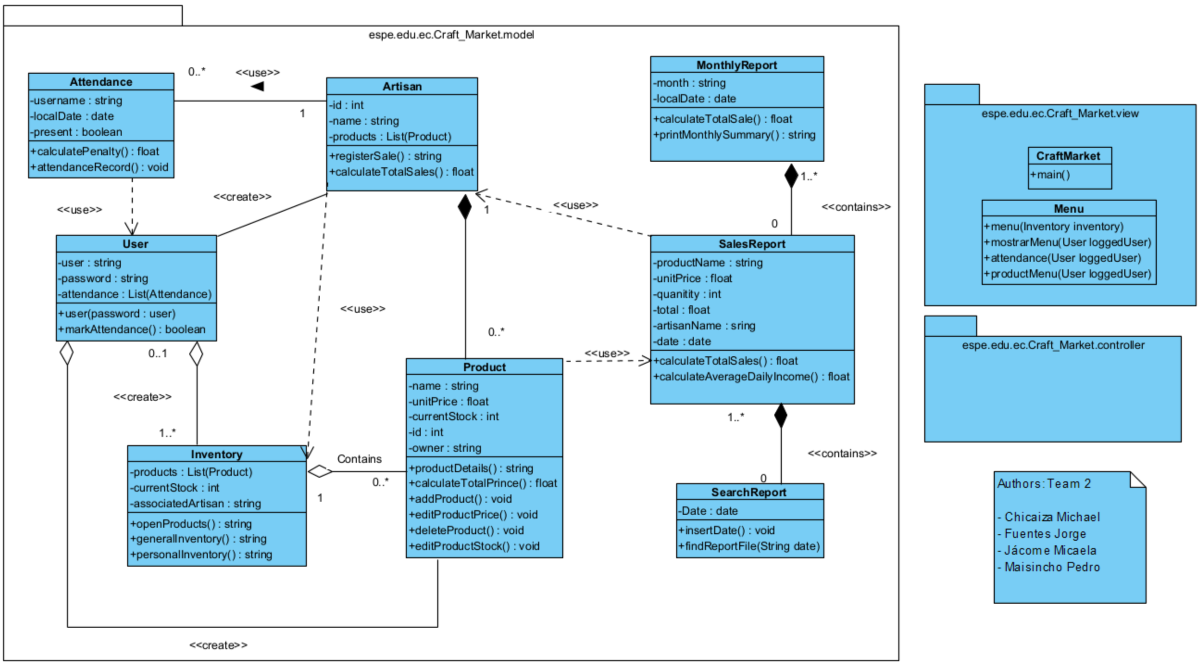
Based on the 10 artisans, they currently manage several manual lists to record attendance, sales, and products. However, this method presents drawbacks such as human error, lack of real-time updating, and difficulty generating accurate reports. Therefore, the proposal is to integrate a digital system that centralizes and automates these processes, making management much more efficient.

2. SRS

ID	TOPIC	As a...	I need to...	So that I can...	Inputs	Output
REQ001	Attendance Registration	Administrator	Record the daily attendance of artisans	Record the daily attendance of artisans	Date, arrival and departure time, artisan's name	Updated attendance record, penalties applied
REQ002	Product Registration	Artisan	Enter new products into the system	Keep the inventory of available products up to date	Product name, quantity, unit price	Product registered in the inventory
REQ003	Sales Registration	Artisan	Record each sale made	Control stock, generate revenue, and track sales statistics	Sold product, quantity, price	Sales record, updated stock
REQ004	Income Control	Administrator	Check total income generated from sales	Have clear visibility of the market's revenue	Date, artisan ID	Total income per period
REQ005	Product History	Artisan	View previously registered products	Review which products were entered and their status	Artisan ID	List of previous products
REQ006	Record Modification	Administrator	Edit incorrect information in sales or attendance	Correct errors without affecting the database	Record ID, new values	Updated record
REQ007	Date filter	Artisian/Administrator	Filter sales or attendance by a date range	Consult specific information for a defined period	Start date, end date	Filtered data by range
REQ008	Record Deletion	Administrator	Delete erroneous or duplicate records	Keep the database clean and accurate	Record ID	Deleted record
REQ009	Access with Username and Password	Artisian/Administrator	Log into the system	Protect information based on user role	Username, password	System access with permissions
REQ010	Low Stock Notification	Artisian	Notify artisans about low stock	Inform the Artisan	Product ID, stock product,	Message

3. Class and UC Diagrams





4. ER (10 tables)/CSV (10files)

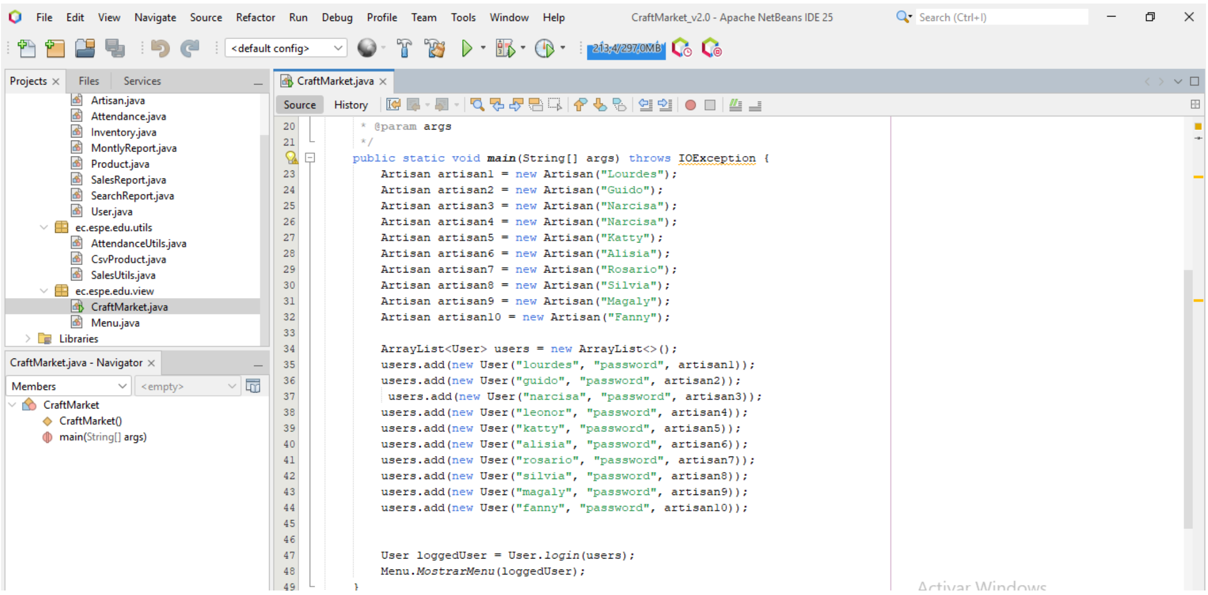
products.csv	CraftMarket_V2.0	yesterday
ventade_2025-05-23.csv	CraftMarket_V2.0	yesterday
ventade_2025-05-24.csv	CraftMarket_V2.0	yesterday
ventade_2025-06-23.csv	CraftMarket_V2.0	yesterday
ventade_2025-06-24.csv	CraftMarket_V2.0	yesterday

1	alexis	2025-05-20	true
2	lourdes	2025-05-01	true
3	lourdes	2025-05-02	true

5. Mockup

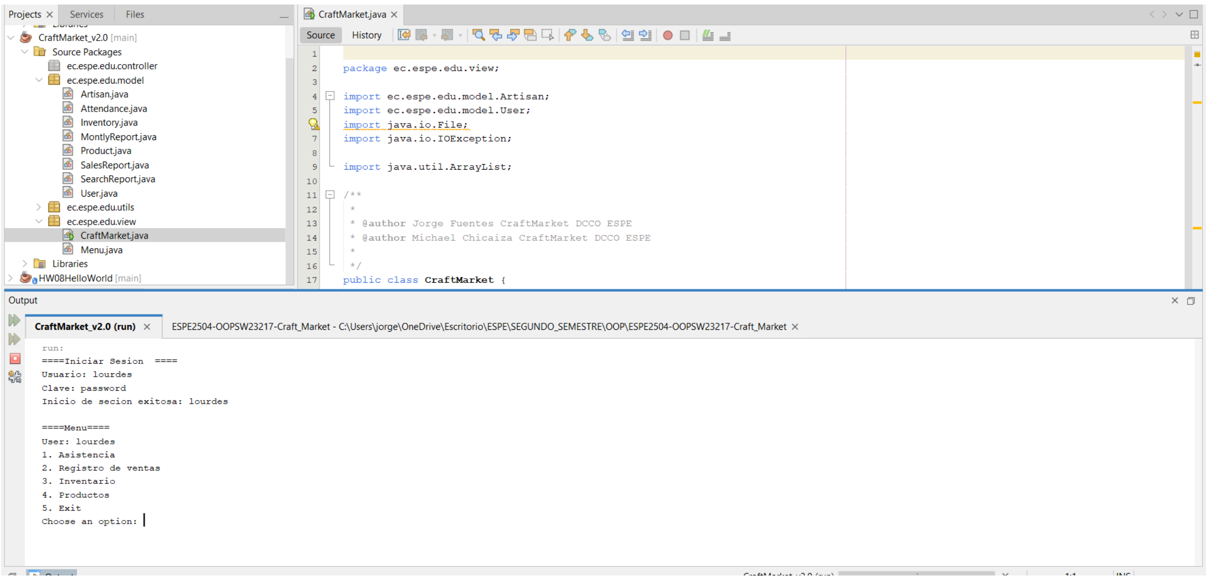
```
====Menu====
User:
1. Asistencia
2. Registro de ventas
3. Inventario
4. Productos
5. Exit
```

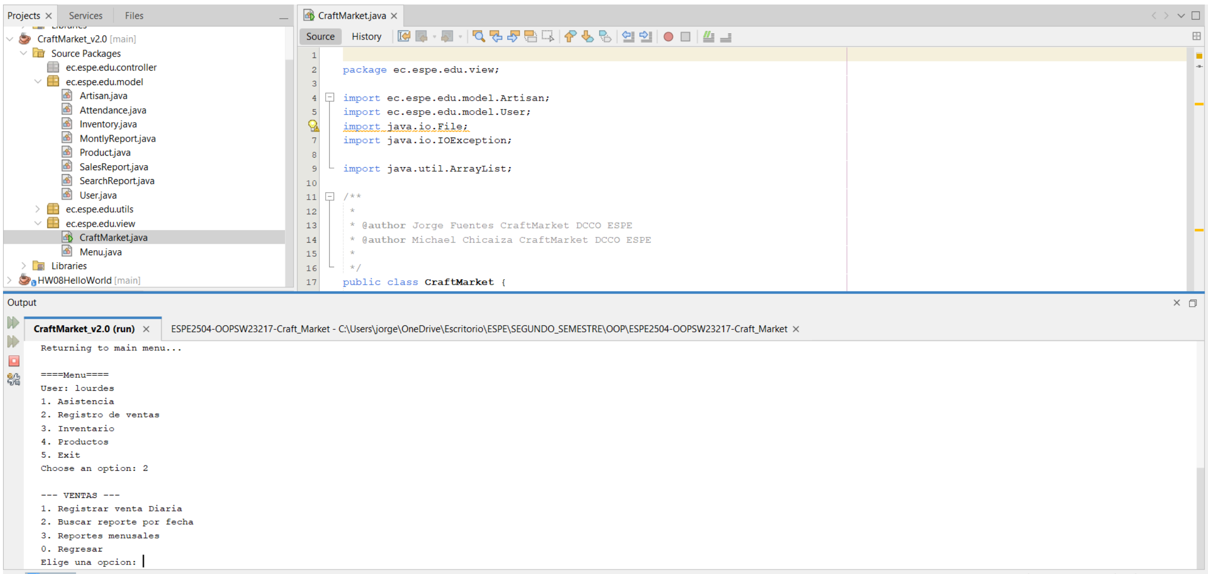
6-7. Clean Code



```java
     * @param args
     */
    public static void main(String[] args) throws IOException {
        Artisan artisan1 = new Artisan("Lourdes");
        Artisan artisan2 = new Artisan("Guido");
        Artisan artisan3 = new Artisan("Narcisa");
        Artisan artisan4 = new Artisan("Narcisa");
        Artisan artisan5 = new Artisan("Katty");
        Artisan artisan6 = new Artisan("Alisia");
        Artisan artisan7 = new Artisan("Rosario");
        Artisan artisan8 = new Artisan("Silvia");
        Artisan artisan9 = new Artisan("Magaly");
        Artisan artisan10 = new Artisan("Fanny");

        ArrayList<User> users = new ArrayList<>();
        users.add(new User("lourdes", "password", artisan1));
        users.add(new User("guido", "password", artisan2));
         users.add(new User("narcisa", "password", artisan3));
        users.add(new User("leonor", "password", artisan4));
        users.add(new User("katty", "password", artisan5));
        users.add(new User("alisia", "password", artisan6));
        users.add(new User("rosario", "password", artisan7));
        users.add(new User("silvia", "password", artisan8));
        users.add(new User("magaly", "password", artisan9));
        users.add(new User("fanny", "password", artisan10));


        User loggedUser = User.login(users);
        Menu.MostrarMenu(loggedUser);
    }
```

8-9-. Functionality





10. persistence (consistency)

Name	Last commit message	Last commit date
..		
products.csv	CraftMarket_V2.0	yesterday
ventade_2025-05-23.csv	CraftMarket_V2.0	yesterday
ventade_2025-05-24.csv	CraftMarket_V2.0	yesterday
ventade_2025-06-23.csv	CraftMarket_V2.0	yesterday
ventade_2025-06-24.csv	CraftMarket_V2.0	yesterday

✔️ Team 3: Bug busters

Project: HealthKeeper
Leader: Daniel Palacios
Github: https://github.com/Carlitos555666/ESPE202550TEAM3

Inspector Team 1: SoftCrafters, Inspector Leader: Andrade Julio

1. . Paillacho Carlos
2. . Palacios Daniel
3. . Quiroz Maria

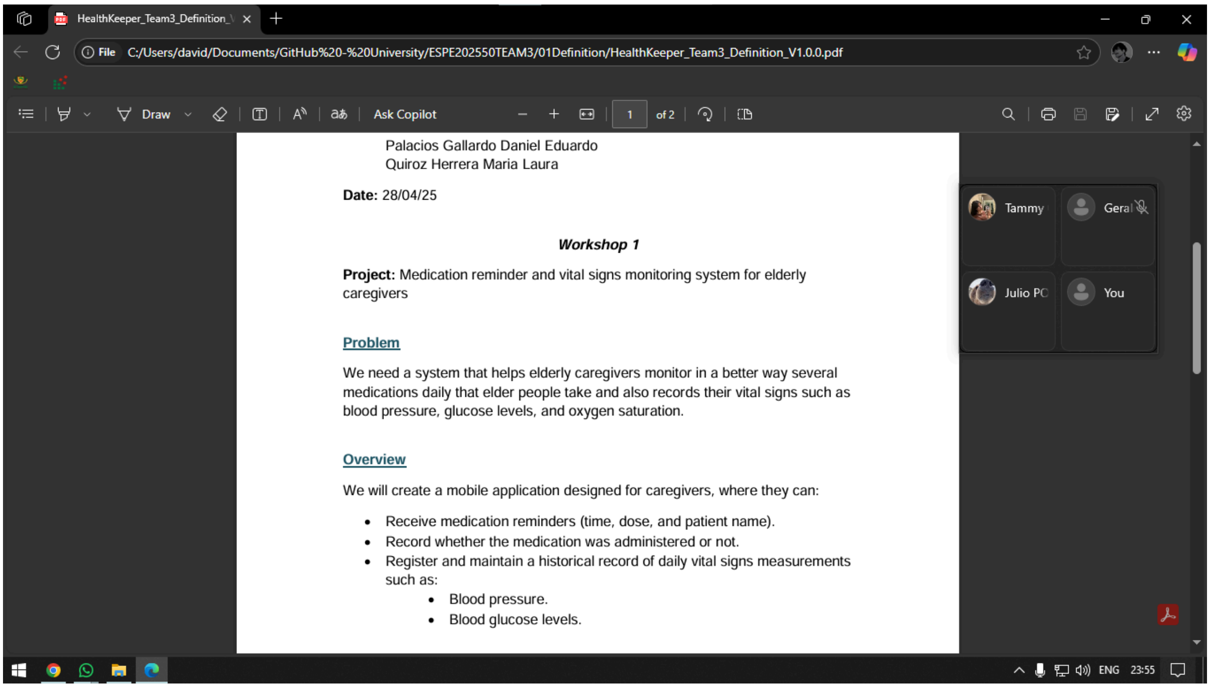
RUBRIC

TOPIC	EVAL	OBSERVATIONS
1. Definition	7.2/10	Important theory is missing for the development of the solution to the problem. The Overview does not follow the
2. SRS	7/10	It does not follow the IEEE Structure. Correct requirement 10, 11 and 12, it 's the same.
3. Class and UC Diagrams	4/10	The use case diagram has a different format than requested, and the class diagram is incomplete because it has no multiplicity. 0/10 UC Diagrams 8/10 Class Diagram
4. ER (10 tables)/CSV (10files)	10/10	They have 10 CSV files containing patient data, medical histories, and related information.
5. Mockup	9/10	The menu design contains too many options, which could have been simplified by distributing them across other menus.
6-7. Clean Code	16/20	The code complies with clean code standards, although its naming could be more descriptive.
8-9-. Functionality	18/20	The code functions correctly; however, it lacks input validation, so entering non-numeric characters in numeric fields or invalid options in the menu causes the program to throw exceptions and crash.
10. Persistence (consistency)	9/10	The program includes file persistence; however, it does not have a dedicated folder for storage, so

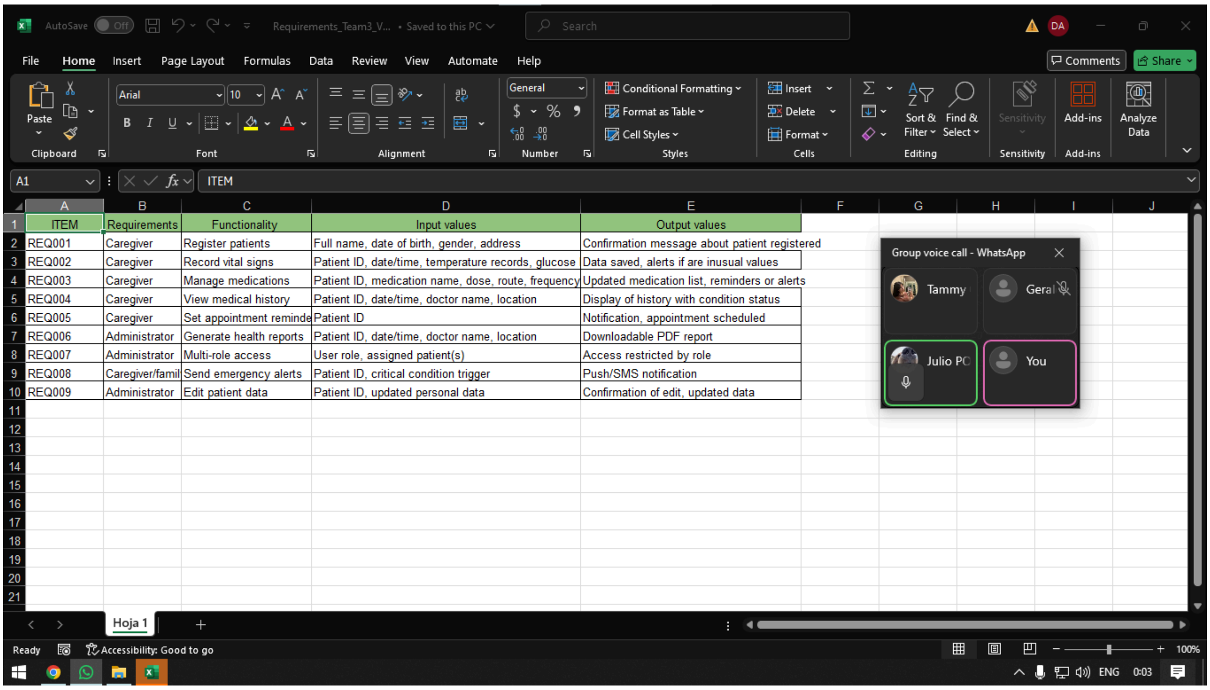
		files are saved in the project's root directory, which indicates a minor lack of organization in the storage structure.
TOTAL	80.2/100	

EVIDENCE

1. Definition

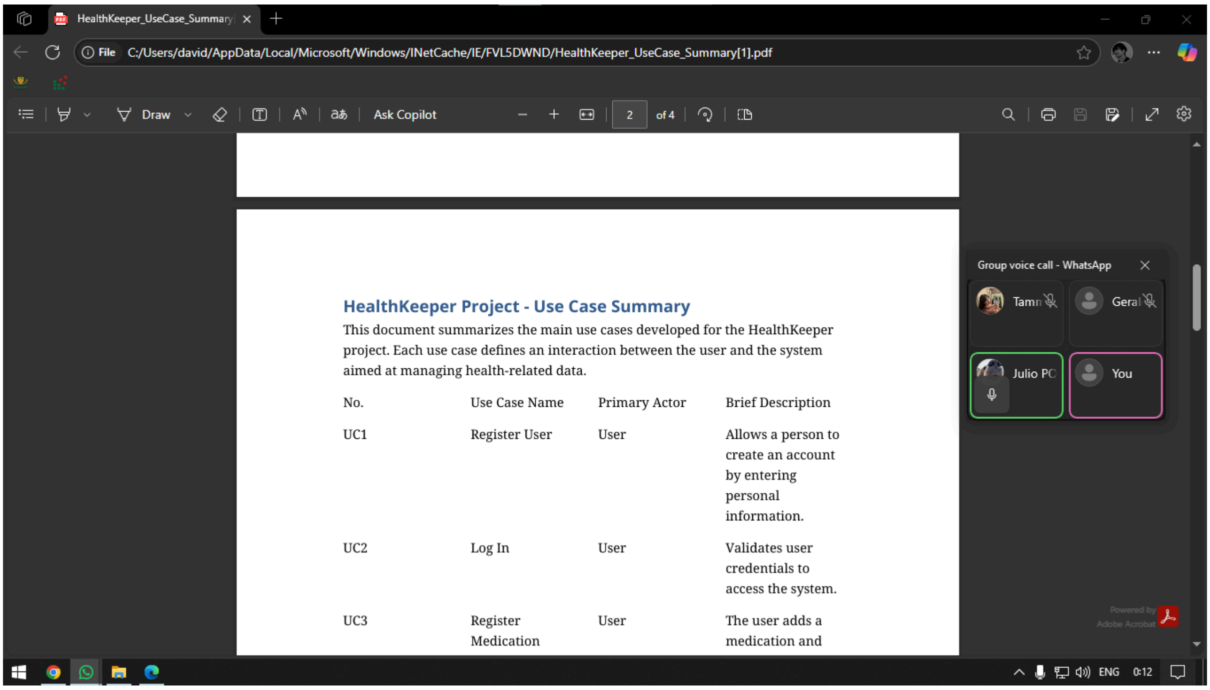


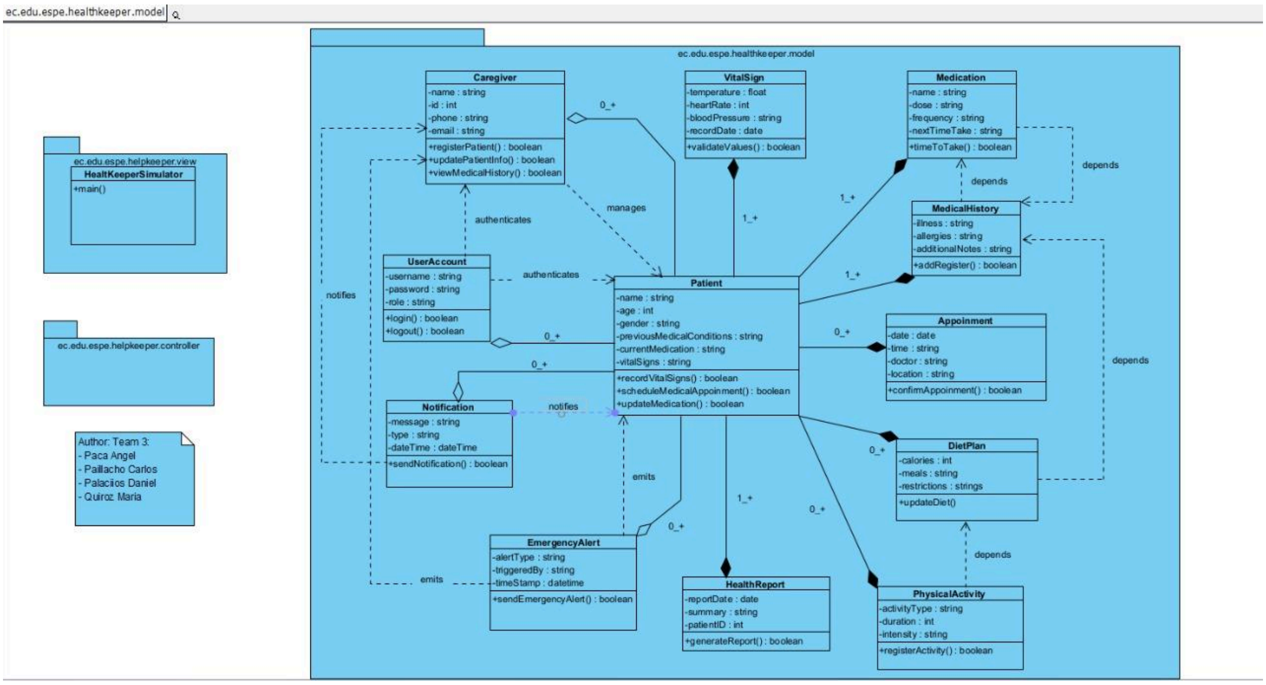
2. SRS



ITEM	Requirements	Functionality	Input values	Output values
REQ001	Caregiver	Register patients	Full name, date of birth, gender, address	Confirmation message about patient registered
REQ002	Caregiver	Record vital signs	Patient ID, date/time, temperature records, glucose	Data saved, alerts if are inusual values
REQ003	Caregiver	Manage medications	Patient ID, medication name, dose, route, frequency	Updated medication list, reminders or alerts
REQ004	Caregiver	View medical history	Patient ID, date/time, doctor name, location	Display of history with condition status
REQ005	Caregiver	Set appointment reminde	Patient ID	Notification, appointment scheduled
REQ006	Administrator	Generate health reports	Patient ID, date/time, doctor name, location	Downloadable PDF report
REQ007	Administrator	Multi-role access	User role, assigned patient(s)	Access restricted by role
REQ008	Caregiver/famil	Send emergency alerts	Patient ID, critical condition trigger	Push/SMS notification
REQ009	Administrator	Edit patient data	Patient ID, updated personal data	Confirmation of edit, updated data

3. Class and UC Diagrams





4. ER (10 tables)/CSV (10files)

appointments	25/5/2025 15:44	Archivo de origen ...	1 KB
build	17/5/2025 20:25	Microsoft Edge H...	4 KB
caregivers	25/5/2025 15:44	Archivo de origen ...	1 KB
dietplans	25/5/2025 15:45	Archivo de origen ...	1 KB
health_report	25/5/2025 15:46	Archivo de origen ...	1 KB
Maria Quiroz_activities	25/5/2025 15:37	Archivo de origen ...	1 KB
medicalHistory	25/5/2025 19:17	Archivo de origen ...	1 KB
medications	25/5/2025 19:17	Archivo de origen ...	1 KB
patients	25/5/2025 19:17	Archivo de origen ...	1 KB
Teresa Robles_activities	25/5/2025 15:46	Archivo de origen ...	1 KB
vitalSigns	25/5/2025 15:46	Archivo de origen ...	1 KB

5. Mockup

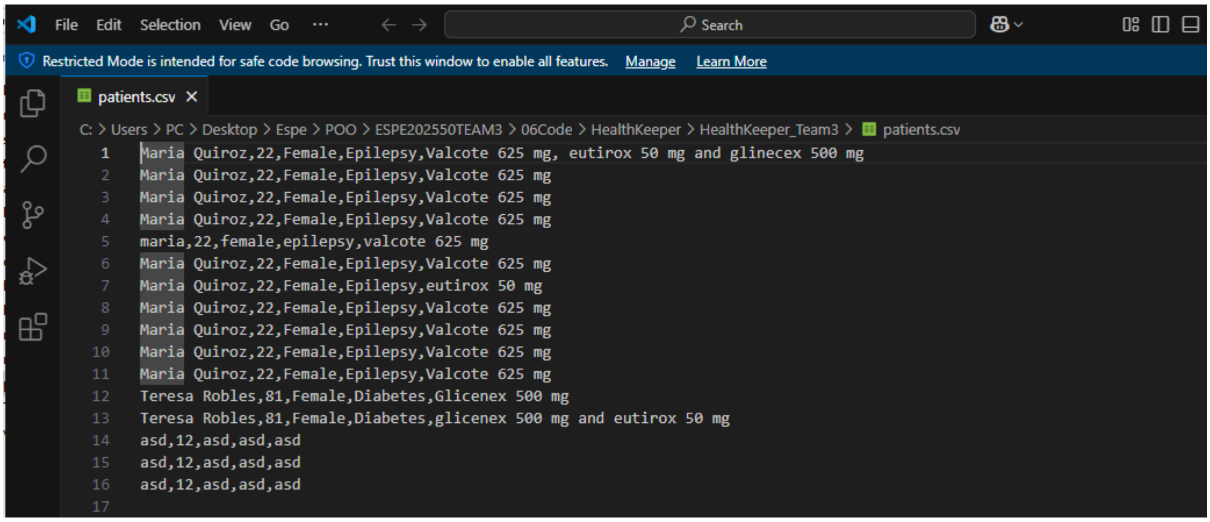
6-7. Clean Code

```
package ec.espe.edu.HealthKeeper.Team3.model;

/**
 *
 * @author Team 3: Pailacho Carlos, Palacios Daniel, Quiroz Maria
 */
import java.io.FileWriter;
import java.io.IOException;

public class DietPlan {
    private String breakfast;
    private String lunch;
    private String dinner;
    private String snacks;

    public DietPlan(String breakfast, String lunch, String dinner, String snacks) {
        this.breakfast = breakfast;
        this.lunch = lunch;
        this.dinner = dinner;
        this.snacks = snacks;
    }

    public String getBreakfast() {
        return breakfast;
    }
```

8-9-. Functionality

```
run:

--- Health Keeper System ---
1. Register patient
2. Record vital signs
3. Schedule medical appointment
4. Add caregiver
5. Assign diet plan
6. Update medication
7. Save vital signs to file
8. Record physical activity
9. Record health report
10. Save all patient data
11. Exit
Choose an option: 1
Enter patient name: asd
Enter age: 12
Enter gender: asd
Enter previous medical conditions: asd
Enter current medication: asd
Patient registered successfully.

--- Health Keeper System ---
1. Register patient
2. Record vital signs
3. Schedule medical appointment
4. Add caregiver
5. Assign diet plan
6. Update medication
7. Save vital signs to file
8. Record physical activity
9. Record health report
10. Save all patient data
11. Exit
Choose an option: 10
All patient data saved successfully.
All patient data saved.

--- Health Keeper System ---
```

10. persistence (consistency)



✔️ Team 4: JSnow

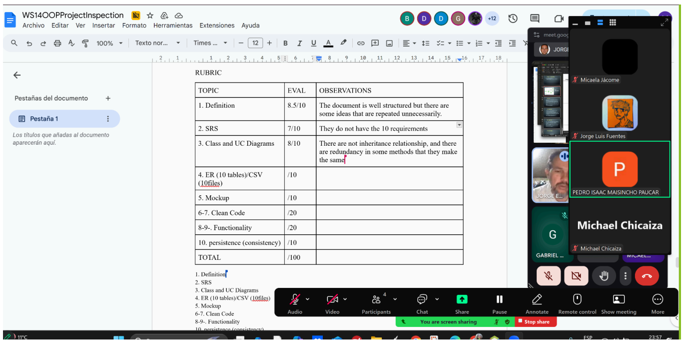
Project: Inventory Management
Leader: Jahir Sivinta
Github: https://github.com/Yesteb/ESPE2504-OOPSW23217-JSnow.git
Inspector Team: 2 Black Mesa

1. .
2. .
3. .

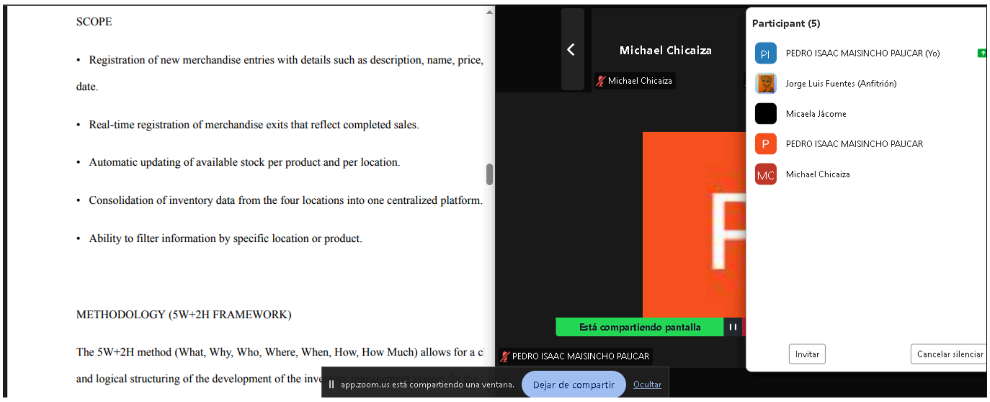
RUBRIC

TOPIC	EVAL	OBSERVATIONS
1. Definition	8.5/10	The document is well structured but there are some ideas that are repeated unnecessarily.
2. SRS	7/10	They do not have the 10 requirements
3. Class and UC Diagrams	7/10	There are not 10 use cases, and the actors are not outside of the system. 6/10 UC Diagrams There are redundancy in the methods of the class diagram, and there are not inheritance relationship 7.5/10 Class Diagram
4. ER (10 tables)/CSV (10 files)	5.5/10	Don't have enough csv files, but the csv files they have are well structured.
5. Mockup	8.5/10	They have more than 5 menu, but some of those are not well structured, and we can’t access to some of that menus
6-7. Clean Code	15.5/20	There are a lot of duplicate code, like in CredentialUserCreate.java, and have some variables that are ambiguous, and so large that it is very difficult to understand.
8-9-. Functionality	18.5/20	Some parameters need to be validated when entering the value of the options
10. persistence (consistency)	9/10	All files that are created are saved correctly, no new files are created when the program is run multiple times, but it is difficult to differentiate between them and the information within them.
TOTAL	79,5/100	

EVIDENCE

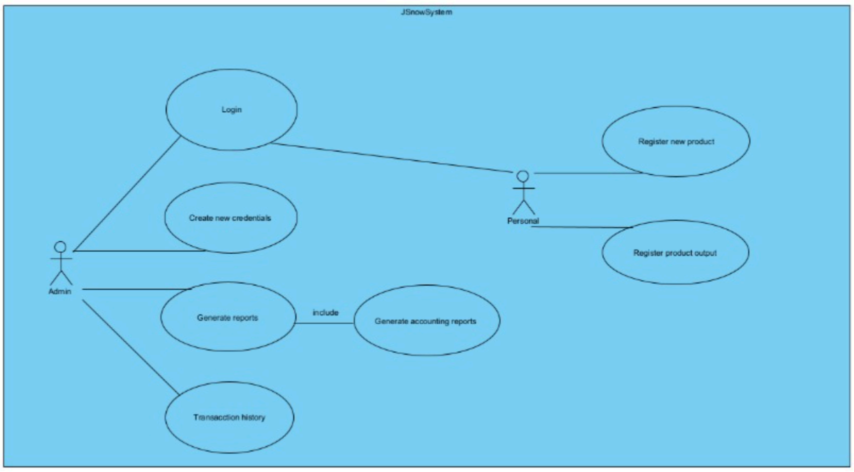


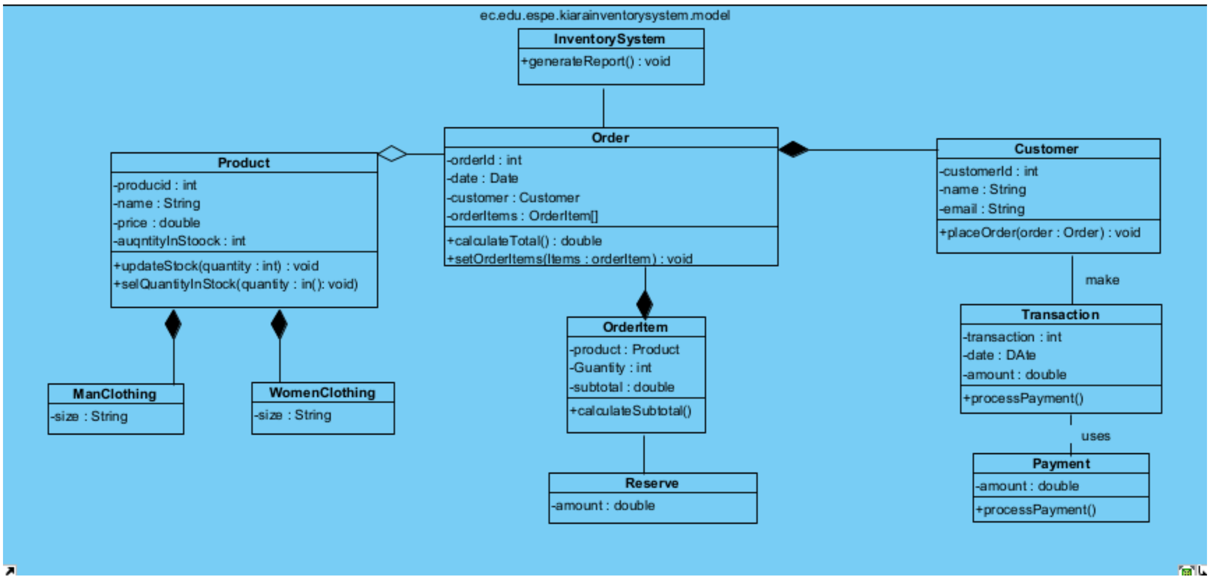
1. Definition



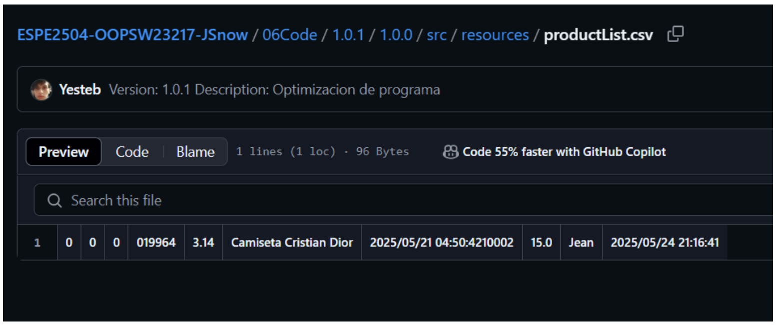
2. SRS

3. Class and UC Diagrams

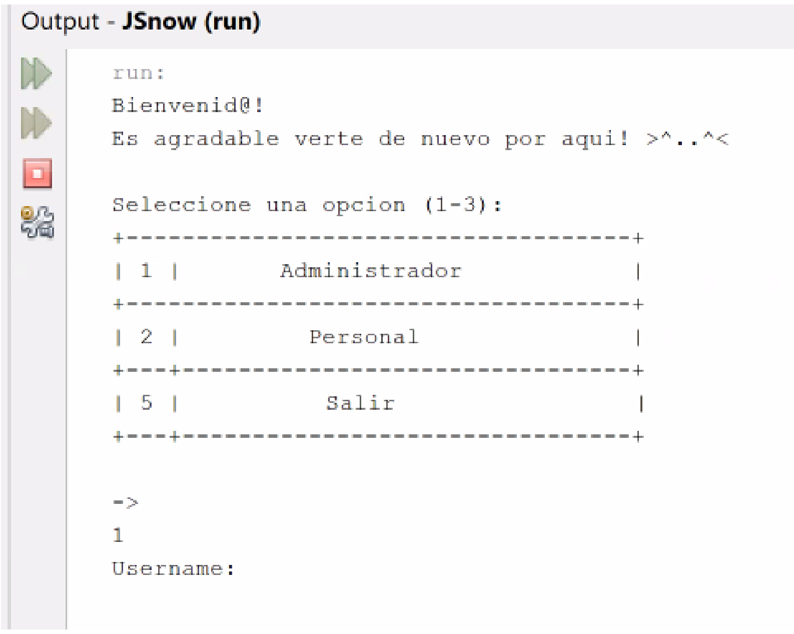




4. ER (10 tables)/CSV (10files)



5. Mockup



```java
if (columns.length > 0 && columns[0].trim().equalsIgnoreCase(id)) {
    System.out.println("Producto encontrado!! >^..^<");
    System.out.println("---------------------------------------");
    System.out.println("Id: " + columns[0]);
    System.out.println("---------------------------------------");
    System.out.println("Precio: " + columns[1]);
    System.out.println("---------------------------------------");
    System.out.println("Name: " + columns[2]);
    System.out.println("---------------------------------------");
    System.out.println("Fecha y Hora del Registro: " + columns[3]);
    findId = true;
    System.console().readLine();
```

```
+---+-----------------------------------+---+-----------------------------------+---+-------------------------------+
| 1 | Registrar nuevo usuario           | 2 | Generar reportes                  | 3 | Salir                         |
+---+--------+--------------------------+---+-----------------------------------+---+-------------------------------+
```

```
==========================================
 | MODULO DE ACTUALIZACION DE CREDENCIALES |
==========================================

Seleccione una opcion (1-3):
+------------------------------------+
| 1 |        Para el administrador    |
+------------------------------------+
| 2 |        Para el Personal         |
+---+-------------+------------------+

+-----------------+
|<--  Regresar    |
+-----------------+

_->
```

```
+---+-----------------------------------+---+-------------------------+---+-----------------------------------+
| 1 | Registrar nuevo producto          | 2 | Buscar producto         | 3 | Registrar salida de productos     |
+---+--------+--------------------------+---+-------------------------+---+-----------------------------------+
| 4 | Salir |                                                                                                 |
+---+--------+----------------------------------------------------------------------------------------------+
```

6-7. Clean Code

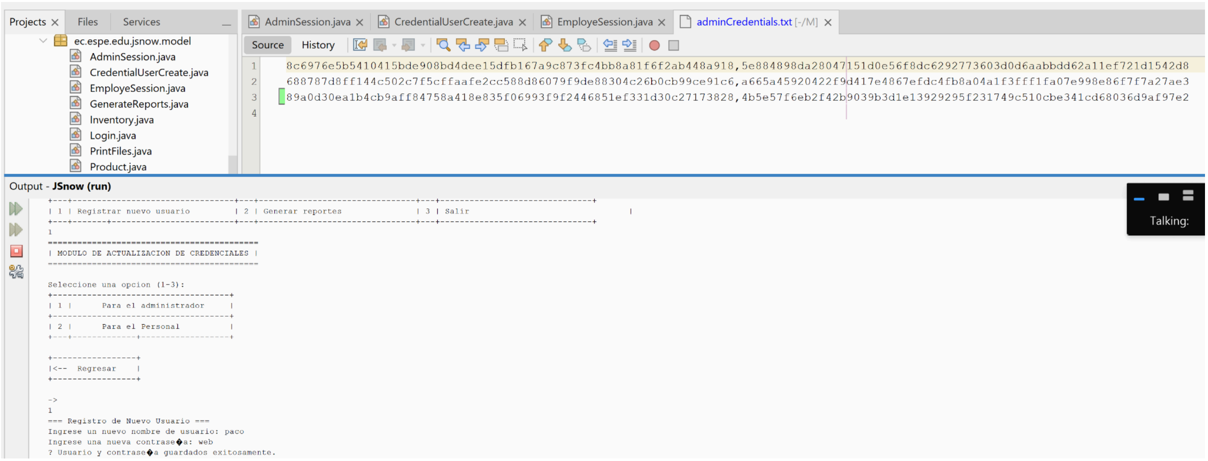
AdminSession.java × | CredentialUserCreate.java ×

Source | History

```java
        String encryptPassword;

        scanner = new Scanner(System.in);

        System.out.println("=== Registro de Nuevo Usuario ===");
        System.out.print("Ingrese un nuevo nombre de usuario: ");
        username = scanner.nextLine();
        System.out.print("Ingrese una nueva contraseña: ");
        password = scanner.nextLine();

        encryptUsername = hash(username);
        encryptPassword = hash(password);

        try (PrintWriter writer = new PrintWriter(new FileWriter("src/resources/employeCredentials.txt", true))) {
            writer.println(encryptUsername + "," + encryptPassword);
            System.out.println("✓ Usuario y contraseña guardados exitosamente.");
            return;
        } catch (IOException e) {
            System.out.println("✗ Error al guardar las credenciales: " + e.getMessage());
            return;
        }
    }

    // Método para encriptar con SHA-256
    public static String hash(String input) {
        try {
            MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
            byte[] hashBytes = messageDigest.digest(input.getBytes());
            StringBuilder hexString = new StringBuilder();

            for (byte b : hashBytes) {
                hexString.append(String.format("%02x", b));
            }

            return hexString.toString();
        } catch (NoSuchAlgorithmException error) {
```

8-9-. Functionality

10. persistence (consistency)



Team 5: Edi-Sons

Project: War Game System
Leader: José Yánez
Github: https://github.com/jgyanez2/ESPE2504-OOPSW23217-Edi-Sons.git
Inspector Team: 3

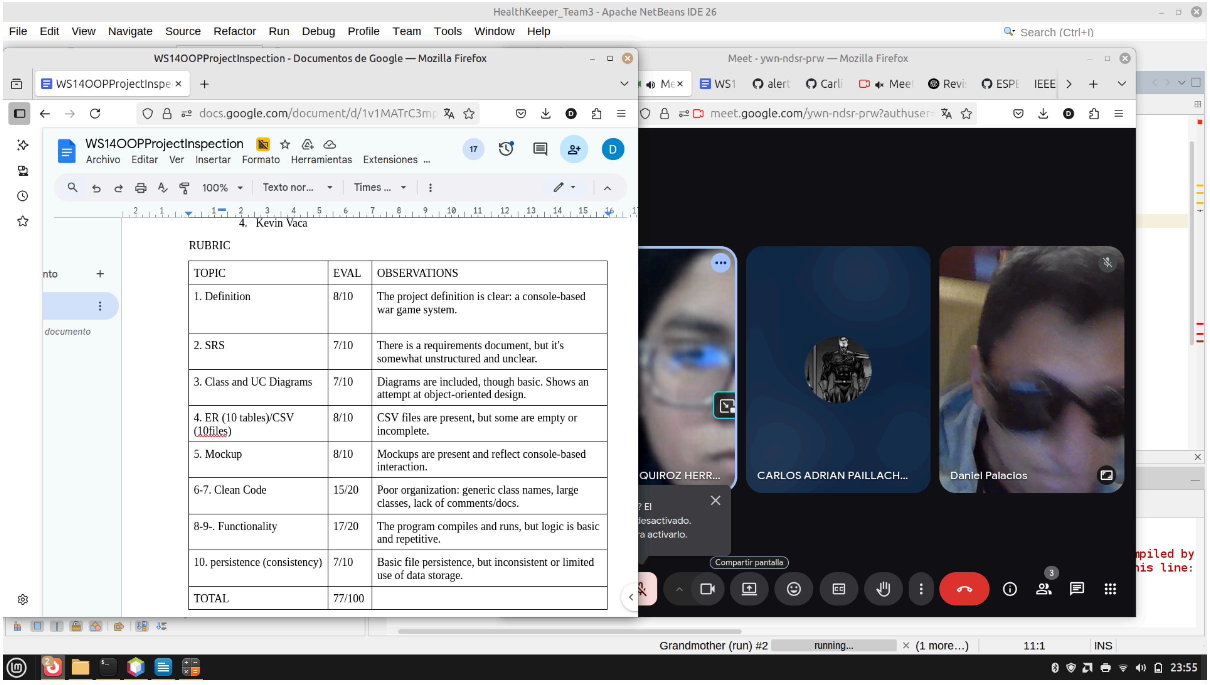
1. José Yánez
2. Mateo Unda
3. Bernardo Suarez
4. Kevin Vaca

RUBRIC

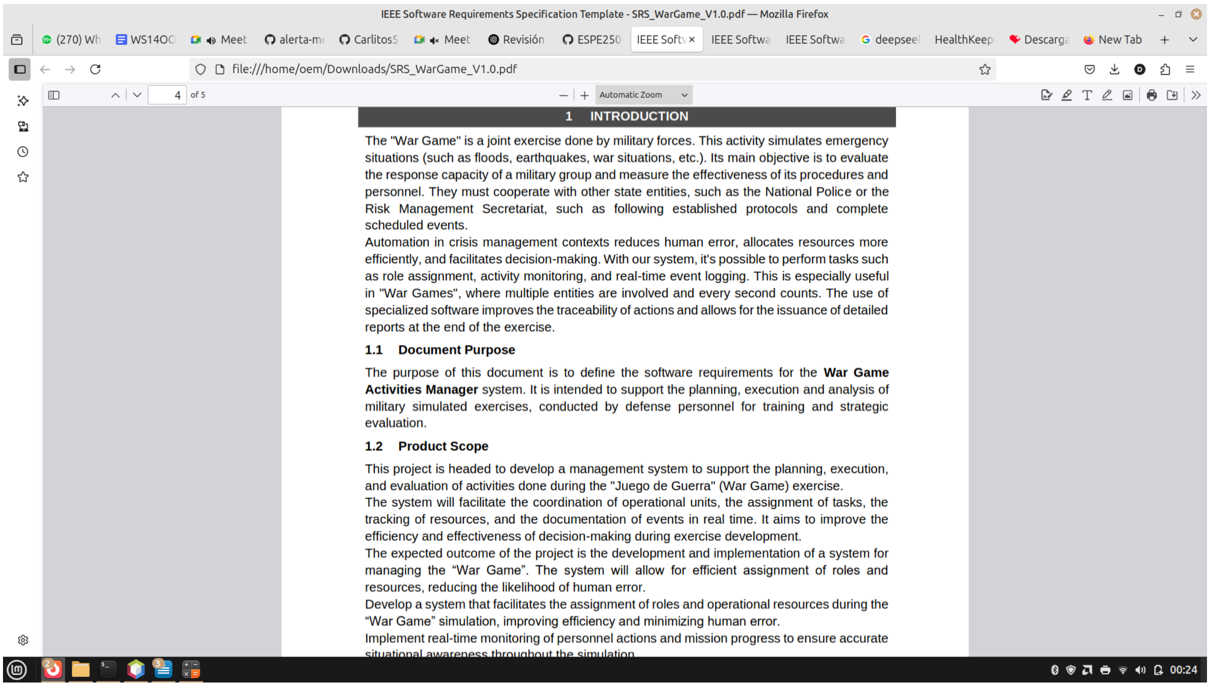
TOPIC	EVAL	OBSERVATIONS
1. Definition	8/10	The project definition is clear: a console-based war game system.
2. SRS	7/10	There is a requirements document, but it's somewhat unstructured and unclear.
3. Class and UC Diagrams	7/10	Diagrams are included, though basic. Shows an attempt at object-oriented design.
4. ER (10 tables)/CSV (10files)	/10	CSV files are present, but some are empty or incomplete.
5. Mockup	/10	
6-7. Clean Code	/20	Poor organization: generic class names, large classes, lack of comments/docs.
8-9-. Functionality	/20	The program compiles and runs, but logic is basic

		and repetitive.
10. persistence (consistency)	/10	Basic file persistence, but inconsistent or limited use of data storage.
TOTAL	/100	

EVIDENCE

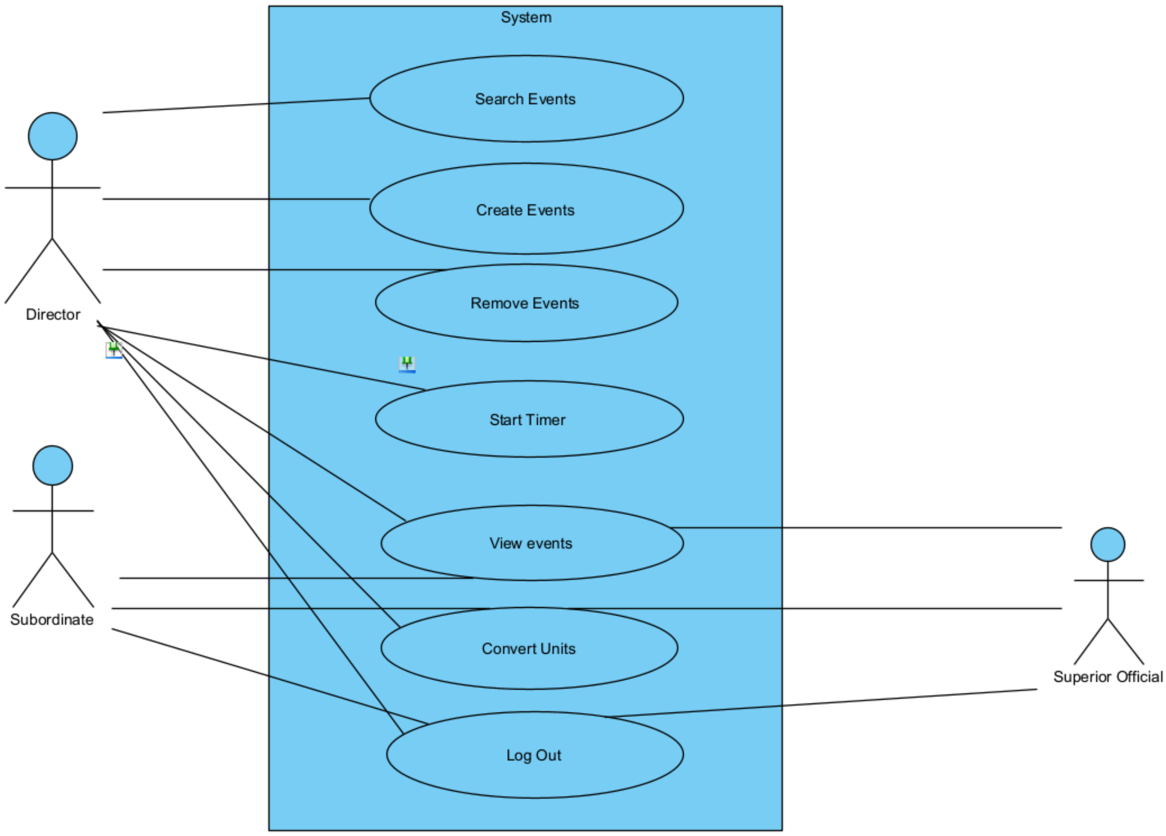


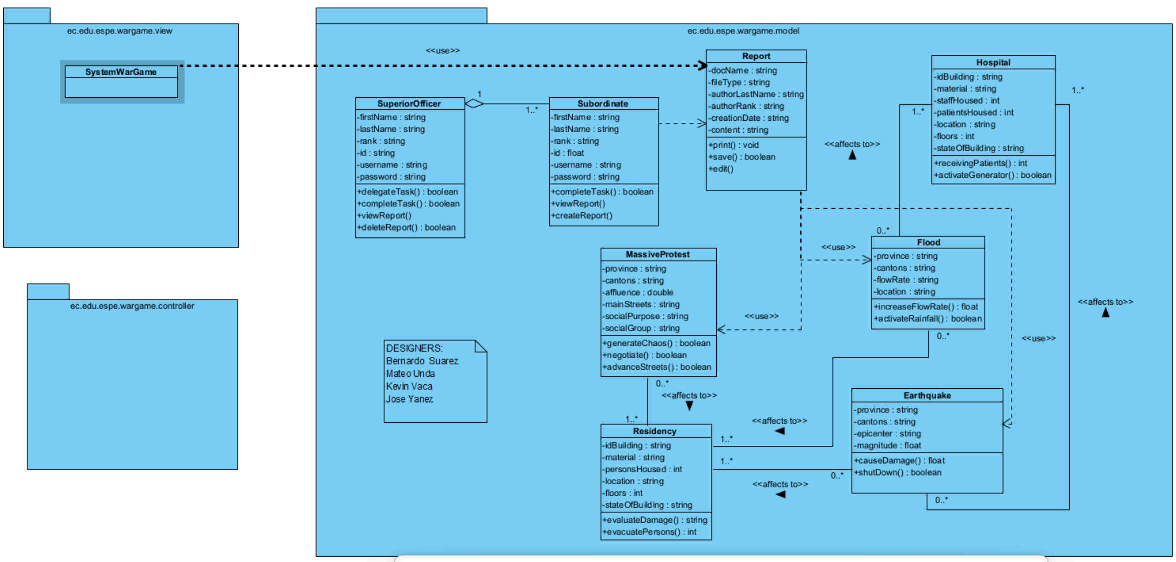
1. Definition



2. SRS

3. Class and UC Diagrams





4. ER (10 tables)/CSV (10files)

5. Mockup

6-7. Clean Code

8-9-. Functionality

10. persistence (consistency)